

Formative Assignment: Advanced Linear Algebra (PCA)

This notebook will guide you through the implementation of Principal Component Analysis (PCA). Fill in the missing code and provide the required answers in the appropriate sections. You will work with a dataset that is Africanized .

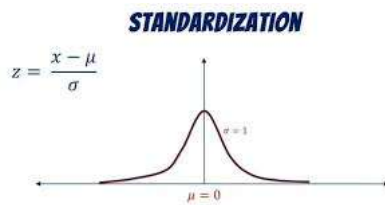
1. Make sure to display outputs for each code cell when submitting.
2. Do not write all your code on one cell
3. Do not use any libraries aside from numpy

Start coding or [generate](#) with AI.

Step 1: Load and Standardize the Data

Before applying PCA, we must standardize the dataset. Standardization ensures that all features have a mean of 0 and a standard deviation of 1, which is essential for PCA. Fill in the code to standardize the dataset.

STRICTLY - Write code that implements standardization based on the image below



```
import numpy as np
import pandas as pd

df = pd.read_csv('life_expectancy.csv')

numeric_df = df.select_dtypes(include=np.number)

numeric_df = numeric_df.fillna(
    numeric_df.mean()
)

data = numeric_df.values

mean = np.mean(data,axis=0)

std = np.std(data,axis=0)

standardized_data = (
    data-mean
)/std

standardized_data[:5]

array([[ -1.64316767e+00,  -1.47126042e+00,   3.69556635e+00,
        -2.07755882e-01,   0.00000000e+00,   5.12035488e-16,
         4.88443356e-01,   0.00000000e+00,   0.00000000e+00,
         1.65252541e-01,   2.71388450e-01,  -5.44655070e-02],
       [ -1.64316767e+00,  -2.48370994e+00,   5.65598154e+00,
        -1.87571694e-01,  -6.80388025e-01,   5.12035488e-16,
        -6.50550521e-01,   0.00000000e+00,   0.00000000e+00,
         1.41679076e-02,   3.52812867e-01,  -1.61274823e-01],
       [ -1.64316767e+00,   4.96935316e-01,  -5.73561128e-01,
        -2.04442654e-01,   2.80566351e-01,  -6.51626995e-01,
         1.78828575e+00,   0.00000000e+00,  -5.14011069e-01,
        -2.30399661e-01,  -2.46570219e-01,  -2.33956066e-01],
       [ -1.64316767e+00,   0.00000000e+00,  -1.76771417e-16,
        -2.08034193e-01,  -1.80221808e-01,   5.12035488e-16,
         1.48659781e-16,   0.00000000e+00,  -1.30208306e+00,
        -2.52532310e-01,  -2.54175490e-01,  -2.51645182e-01],
       [ -1.64316767e+00,   5.24958570e-01,  -7.82539397e-01,
        -7.99050178e-02,  -1.40368706e+00,   5.12035488e-16,
        -9.03455733e-01,   0.00000000e+00,   0.00000000e+00,
        -2.25106166e-01,  -2.50672470e-01,  -2.35681162e-01]])
```

Step 3: Calculate the Covariance Matrix

The covariance matrix helps us understand how the features are related to each other. It is a key component in PCA.

```
n = standardized_data.shape[0]

cov_matrix = (
    1/(n-1)
) * np.dot(
    standardized_data.T,
    standardized_data
)

cov_matrix

array([[ 1.00030257,  0.19724603, -0.12835215,  0.02272376,  0.09106473,
         0.02366179, -0.04796606, -0.00574351,  0.09223576, -0.00595284,
        -0.03729734,  0.01835281],
       [ 0.19724603,  1.00030257, -0.61389794,  0.1290105 ,  0.27539354,
        0.14391667, -0.05631625,  0.1282821 ,  0.50976471, -0.00601071,
        -0.21745304,  0.04440433],
       [-0.12835215, -0.61389794,  1.00030257, -0.11691888, -0.19454004,
        -0.14510616, -0.03583063, -0.10544222, -0.39730647,  0.01264035,
         0.13929237, -0.03932624],
       [ 0.02272376,  0.1290105 , -0.11691888,  1.00030257,  0.10829423,
        -0.03570881, -0.05855746,  0.00800027,  0.09220225,  0.64827634,
         0.20657229,  0.81921604],
       [ 0.09106473,  0.27539354, -0.19454004,  0.10829423,  1.00030257,
        0.32496542,  0.16462394,  0.0024337 ,  0.18002681, -0.08612462,
        -0.17359482, -0.045258 ],
       [ 0.02366179,  0.14391667, -0.14510616, -0.03570881,  0.32496542,
        1.00030257,  0.12243597,  0.18375363,  0.10749634, -0.06988466,
        -0.09061375, -0.07005394],
       [-0.04796606, -0.05631625, -0.03583063, -0.05855746,  0.16462394,
        0.12243597,  1.00030257, -0.01395089, -0.0908762 , -0.07407776,
        -0.09758282, -0.08555888],
       [-0.00574351,  0.1282821 , -0.10544222,  0.00800027,  0.0024337 ,
        0.18375363, -0.01395089,  1.00030257, -0.00154485,  0.03236979,
         0.03480504,  0.02514145],
       [ 0.09223576,  0.50976471, -0.39730647,  0.09220225,  0.18002681,
        0.10749634, -0.0908762 , -0.00154485,  1.00030257, -0.10813487,
        -0.20279647, -0.04982899],
       [-0.00595284, -0.00601071,  0.01264035,  0.64827634, -0.08612462,
        -0.06988466, -0.07407776,  0.03236979, -0.10813487,  1.00030257,
         0.77362158,  0.9529109 ],
       [-0.03729734, -0.21745304,  0.13929237,  0.20657229, -0.17359482,
        -0.09061375, -0.09758282,  0.03480504, -0.20279647,  0.77362158,
         1.00030257,  0.61943509],
       [ 0.01835281,  0.04440433, -0.03932624,  0.81921604, -0.045258 ,
        -0.07005394, -0.08555888,  0.02514145, -0.04982899,  0.9529109 ,
         0.61943509,  1.00030257]])
```

The covariance matrix is used to understand the relationships between variables and features in a dataset. The use of covariance in PCA is important because the directions with the greatest variance are used to calculate the principal components. It also helps identify correlated or redundant features. Without covariance, it would be difficult to detect patterns in the data and perform effective dimensionality reduction.

Step 4: Perform Eigendecomposition

Eigendecomposition of the covariance matrix will give us the eigenvalues and eigenvectors, which are essential for PCA. Fill in the code to compute the eigenvalues and eigenvectors of the covariance matrix.

```
eigenvalues,eigenvectors=np.linalg.eig(
    cov_matrix
)

eigenvalues,eigenvectors

(array([3.1085829 , 2.39330878, 1.29166234, 0.01154654, 0.09086916,
        1.07533941, 0.95789002, 0.35358985, 0.84929374, 0.71974427,
        0.55964843, 0.59215543]),
 array([[ -1.77529829e-02, -1.81867779e-01,  1.72602692e-01,
         1.17918187e-02,  7.39809093e-04,  8.65621501e-02,
        -9.31362195e-01,  7.56998414e-02, -1.67466987e-01,
```

```

5.63292944e-02, -5.54959474e-02, -1.47379334e-01],
[-6.88115078e-02, -5.37589222e-01, 1.74239791e-01,
-1.48466733e-02, -1.52551939e-01, -5.98133170e-02,
5.77720387e-02, -7.68586168e-01, -1.25942343e-01,
-5.63549611e-02, 8.85180328e-02, 1.67027081e-01],
[ 5.57802927e-02, 4.86802172e-01, -1.11254274e-01,
-1.18999445e-02, -4.16278593e-02, 5.99340048e-02,
-1.40485046e-01, -5.46918888e-01, 3.29326061e-01,
2.39282161e-01, -4.64307792e-01, -2.04740001e-01],
[ 4.10157815e-01, -2.38071348e-01, -2.78185837e-02,
2.14954524e-01, -5.42684517e-01, 2.05105185e-01,
1.13686710e-01, 1.25859475e-01, 8.93285232e-02,
5.15773764e-01, 1.38738338e-01, -2.63671943e-01],
[-9.35182336e-02, -3.17424235e-01, -4.27258458e-01,
-1.35809931e-02, 6.32642644e-02, 2.97320736e-01,
-1.14377351e-01, 1.22155628e-01, 3.60746674e-01,
1.68050148e-01, -2.43518180e-01, 6.08009605e-01],
[-9.42798876e-02, -2.22728833e-01, -5.55856432e-01,
-1.35317615e-03, -2.32165839e-02, -2.13759553e-01,
-1.19843339e-01, -5.82238087e-02, 4.04947249e-01,
-3.37037337e-01, 2.83577340e-01, -4.58854903e-01],
[-8.01529154e-02, 1.60382520e-03, -5.48265665e-01,
-1.03042920e-02, -3.31183949e-02, 3.49261662e-01,
8.78715677e-02, -7.81629554e-02, -6.91355344e-01,
-5.65652072e-02, -2.00092898e-01, -1.86416528e-01],
[ 9.05152342e-03, -1.17696440e-01, -2.32313858e-01,
8.04244342e-05, 1.91888581e-02, -8.09345730e-01,
-2.23898810e-02, 8.08941421e-02, -2.19609656e-01,
3.94530480e-01, -2.45890275e-01, 7.27822971e-02],
[-1.02461060e-01, -4.27881582e-01, 2.70554479e-01,
-1.55166214e-03, 6.57621237e-02, 4.54127979e-02,
2.20944517e-01, 2.03722457e-01, 1.30190909e-01,
-1.81121285e-01, -6.69221483e-01, -3.72691113e-01],
[ 5.48567279e-01, -7.14873728e-02, -5.72298002e-02,
6.33351082e-01, 5.04937804e-01, -9.14094667e-03,
-1.37333286e-02, -1.06303626e-01, -3.30167214e-02,
-1.38809494e-01, -3.86621072e-02, 3.41600196e-02],
[ 4.37959586e-01, 1.15085298e-01, -5.44888843e-02,
-8.29595585e-02, -5.47554893e-01, -1.61916360e-01,
-1.07167528e-01, 5.86243021e-02, -3.59750798e-02,
-5.56252982e-01, -2.49717778e-01, 2.66589455e-01],
[ 5.44274957e-01, -1.36413332e-01, -3.40921266e-02,
-7.38229453e-01, 3.39673924e-01, 6.15578316e-02,
2.02200731e-02, -3.71285711e-02, -1.75015591e-03,
9.05167669e-02, 4.82408246e-02, -8.67901376e-02]]))

```

Start coding or [generate](#) with AI.

▼ Step 5: Sort Principal Components

Sort the eigenvectors based on their corresponding eigenvalues in descending order. The higher the eigenvalue, the more important the eigenvector. Complete the code to sort the eigenvectors and print the sorted components.

[How Is Explained Variance Used In PCA?](#)

```

sorted_indices=np.argsort(
eigenvalues
)[::-1]

sorted_eigenvalues=(
eigenvalues[
sorted_indices
])

sorted_eigenvectors=(
eigenvectors[
:,
sorted_indices
]
)

sorted_eigenvectors

array([[ -1.77529829e-02,  -1.81867779e-01,   1.72602692e-01,
         8.65621501e-02,  -9.31362195e-01,  -1.67466987e-01,
         5.63292944e-02,  -1.47379334e-01,  -5.54959474e-02,
         7.56998414e-02,   7.39809093e-04,   1.17918187e-02],

```

```

[-6.88115078e-02, -5.37589222e-01, 1.74239791e-01,
-5.98133170e-02, 5.77720387e-02, -1.25942343e-01,
-5.63549611e-02, 1.67027081e-01, 8.85180328e-02,
-7.68586168e-01, -1.52551939e-01, -1.48466733e-02],
[ 5.57802927e-02, 4.86802172e-01, -1.11254274e-01,
5.99340048e-02, -1.40485046e-01, 3.29326061e-01,
2.39282161e-01, -2.04740001e-01, -4.64307792e-01,
-5.46918888e-01, -4.16278593e-02, -1.18999445e-02],
[ 4.10157815e-01, -2.38071348e-01, -2.78185837e-02,
2.05105185e-01, 1.13686710e-01, 8.93285232e-02,
5.15773764e-01, -2.63671943e-01, 1.38738338e-01,
1.25859475e-01, -5.42684517e-01, 2.14954524e-01],
[-9.35182336e-02, -3.17424235e-01, -4.27258458e-01,
2.97320736e-01, -1.14377351e-01, 3.60746674e-01,
1.68050148e-01, 6.08009605e-01, -2.43518180e-01,
1.22155628e-01, 6.32642644e-02, -1.35809931e-02],
[-9.42798876e-02, -2.22728833e-01, -5.55856432e-01,
-2.13759553e-01, -1.19843339e-01, 4.04947249e-01,
-3.37037337e-01, -4.58854903e-01, 2.83577340e-01,
-5.82238087e-02, -2.32165839e-02, -1.35317615e-03],
[-8.01529154e-02, 1.60382520e-03, -5.48265665e-01,
3.49261662e-01, 8.78715677e-02, -6.91355344e-01,
-5.65652072e-02, -1.86416528e-01, -2.00092898e-01,
-7.81629554e-02, -3.31183949e-02, -1.03042920e-02],
[ 9.05152342e-03, -1.17696440e-01, -2.32313858e-01,
-8.09345730e-01, -2.23898810e-02, -2.19609656e-01,
3.94530480e-01, 7.27822971e-02, -2.45890275e-01,
8.08941421e-02, 1.91888581e-02, 8.04244342e-05],
[-1.02461060e-01, -4.27881582e-01, 2.70554479e-01,
4.54127979e-02, 2.20944517e-01, 1.30190909e-01,
-1.81121285e-01, -3.72691113e-01, -6.69221483e-01,
2.03722457e-01, 6.57621237e-02, -1.55166214e-03],
[ 5.48567279e-01, -7.14873728e-02, -5.72298002e-02,
-9.14094667e-03, -1.37333286e-02, -3.30167214e-02,
-1.38809494e-01, 3.41600196e-02, -3.86621072e-02,
-1.06303626e-01, 5.04937804e-01, 6.33351082e-01],
[ 4.37959586e-01, 1.15085298e-01, -5.44888843e-02,
-1.61916360e-01, -1.07167528e-01, -3.59750798e-02,
-5.56252982e-01, 2.66589455e-01, -2.49717778e-01,
5.86243021e-02, -5.47554893e-01, -8.29595585e-02],
[ 5.44274957e-01, -1.36413332e-01, -3.40921266e-02,
6.15578316e-02, 2.02200731e-02, -1.75015591e-03,
9.05167669e-02, -8.67901376e-02, 4.82408246e-02,
-3.71285711e-02, 3.39673924e-01, -7.38229453e-01]]))

```

How Is Explained Variance Used In PCA?

Explained variance measures how much of the dataset's total variance is captured by each principal component. In PCA, it is used to rank components according to their importance and to determine how many components should be retained. Components with higher explained variance preserve more information from the original dataset. In this analysis, the cumulative explained variance was used to select 8 principal components that retain at least 90% of the total variance.

▼ Step 6: Project Data onto Principal Components

Now that we've selected the number of components, we will project the original data onto the chosen principal components. Fill in the code to perform the projection.

```

explained_variance = (
    sorted_eigenvalues /
    np.sum(sorted_eigenvalues)
)

cumulative = np.cumsum(
    explained_variance
)

num_components = np.argmax(
    cumulative >= 0.90
) + 1

principal_components = (
    sorted_eigenvectors[
        :,
        :num_components
    ]
)

```

```

reduced_data = np.dot(
    standardized_data,
    principal_components
)

reduced_data[:5]

```

```

array([[ 0.39205279,  2.965876 , -1.23552025,  0.24643209,  0.91306113,
         1.30614619,  0.56102422, -0.71375475],
       [ 0.62892145,  4.70857356, -0.70757136, -0.18985274,  0.55038047,
         2.62532308,  1.01368312, -1.46509799],
       [-0.53805778,  0.10004363, -0.97012067,  0.60330712,  1.7308597 ,
        -1.44359482,  0.04342322,  0.77099953],
       [-0.29270261,  0.98583913, -0.51622983, -0.26964711,  1.26527789,
         0.03998135,  0.15935325,  0.61817324],
       [-0.2412367 ,  0.11821729,  1.02678465, -0.94167538,  1.76791497,
         0.07930963, -0.3860377 , -0.22796052]])

```

▼ Step 7: Output the Reduced Data

Finally, display the reduced data obtained by projecting the original dataset onto the selected principal components.

```

print(
    "Reduced Data Shape:",
    reduced_data.shape
)

reduced_data[:5]

```

```

Reduced Data Shape: (3306, 8)
array([[ 0.39205279,  2.965876 , -1.23552025,  0.24643209,  0.91306113,
         1.30614619,  0.56102422, -0.71375475],
       [ 0.62892145,  4.70857356, -0.70757136, -0.18985274,  0.55038047,
         2.62532308,  1.01368312, -1.46509799],
       [-0.53805778,  0.10004363, -0.97012067,  0.60330712,  1.7308597 ,
        -1.44359482,  0.04342322,  0.77099953],
       [-0.29270261,  0.98583913, -0.51622983, -0.26964711,  1.26527789,
         0.03998135,  0.15935325,  0.61817324],
       [-0.2412367 ,  0.11821729,  1.02678465, -0.94167538,  1.76791497,
         0.07930963, -0.3860377 , -0.22796052]])

```

▼ Step 8: Visualize Before and After PCA

Now, let's plot the original data and the data after PCA to compare the reduction in dimensions visually.

```

import matplotlib.pyplot as plt

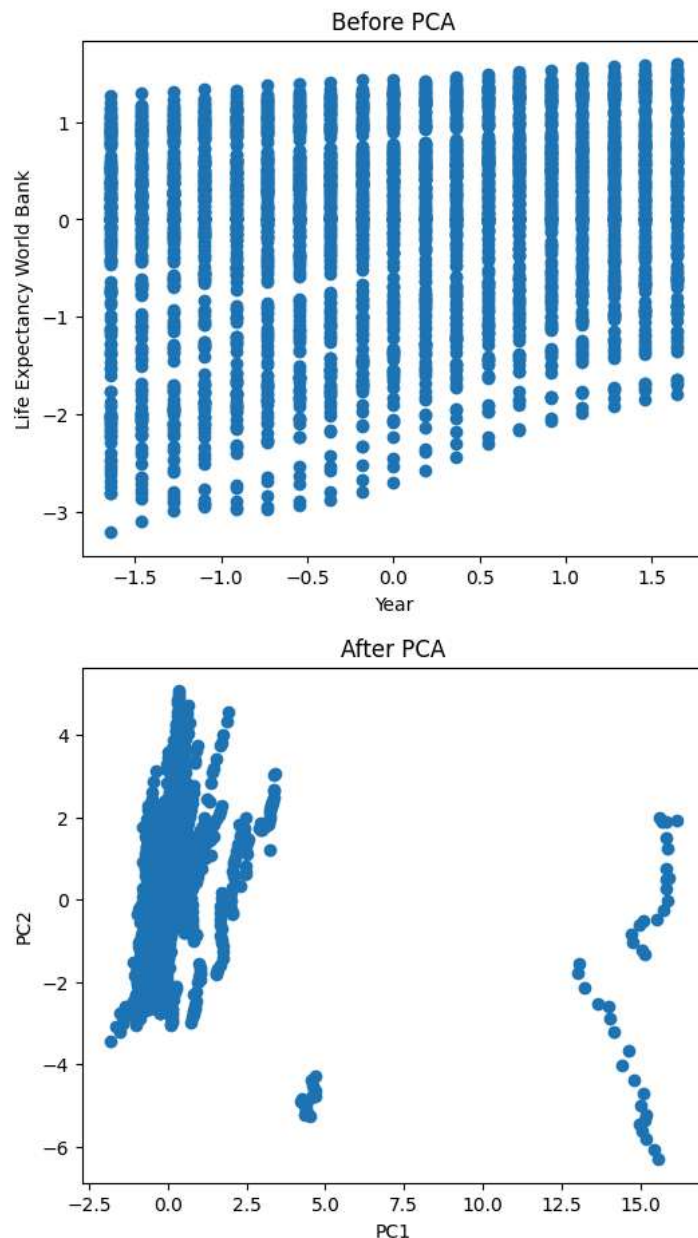
plt.figure(figsize=(6,5))
plt.scatter(
    standardized_data[:, 0],
    standardized_data[:, 1]
)

plt.title("Before PCA")
plt.xlabel(numeric_df.columns[0])
plt.ylabel(numeric_df.columns[1])
plt.show()

plt.figure(figsize=(6,5))
plt.scatter(
    reduced_data[:, 0],
    reduced_data[:, 1]
)

plt.title("After PCA")
plt.xlabel("PC1")
plt.ylabel("PC2")
plt.show()

```



1. Interpretation of the Visual of before and after PCA

Before PCA, the data appears widely scattered across the original standardized features making patterns difficult to observe. After PCA, the data is projected onto principal components that capture the greatest variance. The transformed plot shows a more organized structure and clearer separation between observations. This indicates that PCA successfully compressed the dataset while preserving its most important information.

2. Why we selected 8 principle components and what are the tradeoffs?

Eight principal components were selected because they retain at least 90% of the total variance in the dataset. This reduces the dimensionality from 12 features to 8 while preserving most of the information. The main benefits are reduced computational complexity and lower feature redundancy. The tradeoff is that approximately 10% of the variance is discarded which may contain useful but less significant patterns.

3. What information is lost when reducing dimensions?

In this life expectancy dataset, dimensionality reduction may remove subtle information related to economic activity, population pressure, corruption, unemployment, health expenditure, disease prevalence, and Carbon-dioxide emissions. Small country-specific or year-specific variations may also become less apparent. Relationships represented by lower-variance components are lost during the reduction process. However, the most important patterns influencing life expectancy are retained by the selected principal components.

Please ensure this tracking sheet is completed as a group, as it will be used to monitor your individual participation within the team.						
Team Members	Role/ Responsibility	Email				
Julius Kate Lorna Iriza	task 1 & 3	j.iriza@alustudent.com				
Kellia Kamikazi	task 2 & 3	k.kamikazi@alustudent.com				
Task Allocation and Tracker						
	Task Allocated	Assigned Member	Deadline	Completion Status (Not Started/In Progress/Completed/Did not participate)	Reviewed by Team? (Yes/No)	Comments
	Implement PCA (covariance matrix, eigendecomposition, projection)	Julius Iriza Kate Lorna	10/06/2026	Completed	Yes	
	Select principal components using explained variance and analyze tradeoffs and information loss	Kellia Kamikazi	10/06/2026	Completed	Yes	
	Optimize implementation for large datasets	Julius Iriza Kate Lorna and Kellia Kamikazi	10/06/2026	Completed	Yes	
	Final review	Julius Iriza Kate Lorna and Kellia Kamikazi	10/06/2026	Completed	Yes	
Meeting Attendance Log						
Meeting Date	Agenda	Facilitator	Attendees	Key Discussion Points	Next Steps	

07/06/2026	Discuss the assignment requirements and divide tasks among team members	Kellia Kamikazi	- Julius Iriza Kate Lorna - Kellia Kamikazi	- Reviewed assignment instructions - agreed that Julius would lead Task 1, Kellia would lead Task 2 and both members would collaborate on Task 3. - Discussed deadlines and expected outputs.	Complete assigned tasks and share progress before final review.	
09/06/2026	Review completed work and prepare final submission	Julius Iriza Kate Lorna	- Julius Iriza Kate Lorna - Kellia Kamikazi	Reviewed PCA implementation and confirmed all notebook outputs were displayed.	submit the final deliverables	